April 20 2026

# Predicting resolution time of NYC 311 complaints

Group 6: Ryan Miner, Myra Mulongoti, Obumneme Umeonwuka
*CSCI-635.02: Introduction to Machine Learning*
*Rochester Institute of Technology*

# Introduction

1

# Background

- **NYC 311** is New York City's non-emergency service system where residents report issues like noise, sanitation, street conditions, and housing complaints (via phone, web, and mobile).
- Each 311 record includes **what happened** (complaint type/descriptor), **who handled it** (agency), **where** (borough/zip/lat–long), and **when** (created/closed timestamps).
- **NYC Open Data** is the city's public portal that publishes datasets (including 311) for transparency and analysis; data is accessible through downloads and an API.

# Problem & Motivation

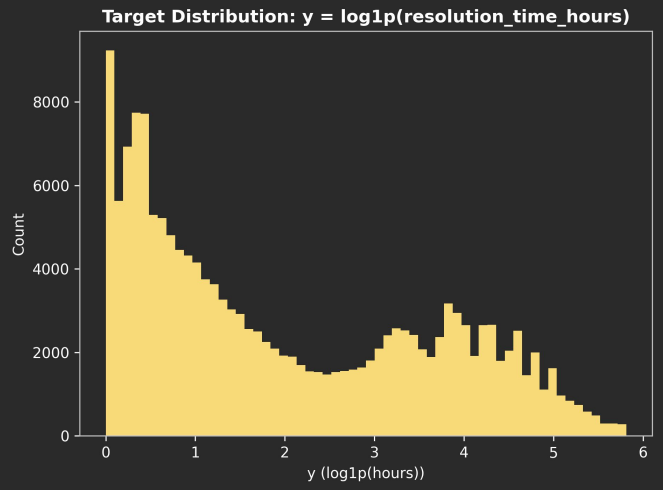
- NYC 311 is a major public service channel; resolution time affects trust and operations.
- **Goal:** Predict resolution time from complaint metadata at creation time. (**Regression**)
- **Why it matters:** better staffing, and expectation-setting for residents.

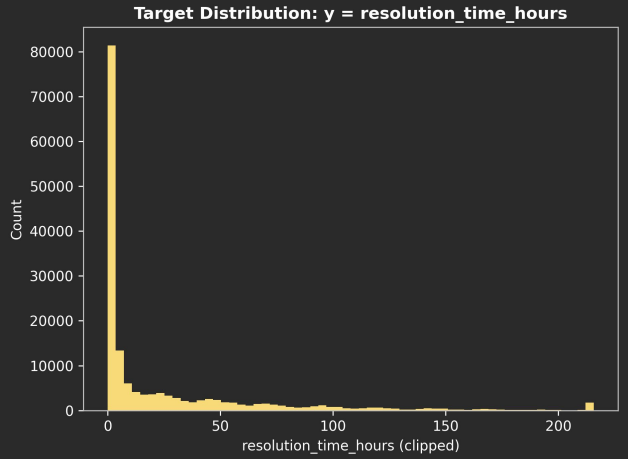
# Dataset & Prediction Task Overview

- Dataset: **NYC Open Data — 311 Service Requests (2020–present)** (live source)
- We convert each complaint into a training example with:
  - **Inputs**: agency, complaint type, descriptor, borough, location, time-of-day, etc.
  - **Target**:
    - `resolution_time_hours = closed_date - created_date`
- We use a **frozen snapshot** to ensure fair model comparison.

# Approach Overview

- Standard ML workflow:
  - clean → features → model → evaluate
- Fair evaluation using **5-fold cross-validation**
- Metrics: **MAE** and **RMSE** on the transformed target (`log1p(hours)`)



# Methods

2

# Dataset Details (311 Requests)

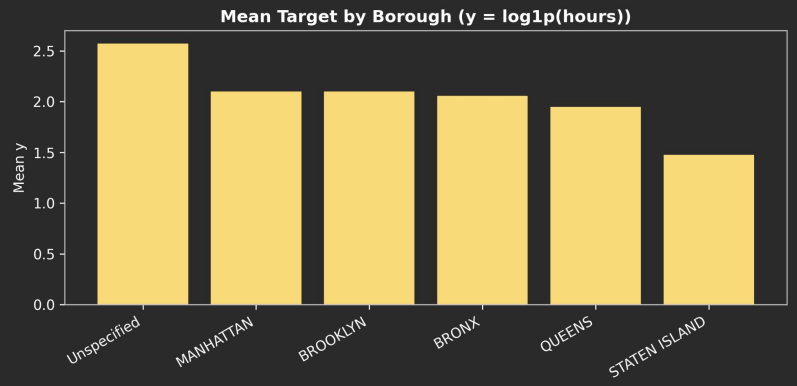
- Snapshot used: processed_2026-03-13_cv5
- Rows used for modeling: N = **160,945**
- Only **closed** cases included (must have created_date and closed_date)
- Target transform: **log1p(resolution_time_hours)**
- 5-fold CV (**≈128,756 train / 32,189 test per fold**)

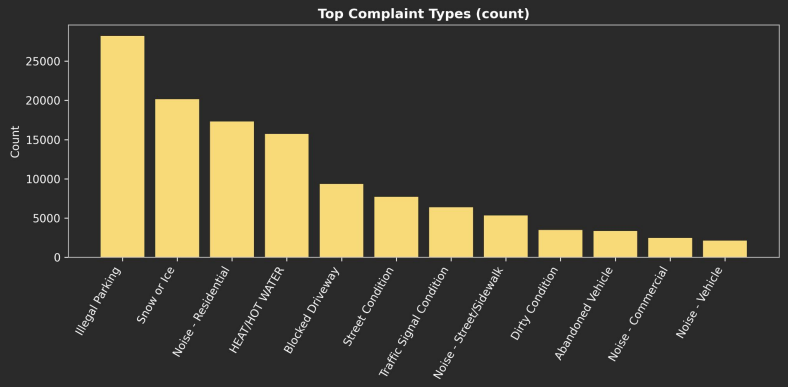
# Data Preprocessing: Cleaning & Filtering

- **Filter to valid examples**:
  - created_date and closed_date present
  - closed_date ≥ created_date
  - remove duplicates by unique_key (keep one record)
- **Compute target**: resolution_time_hours
- **Output**: interim cleaned dataset saved before modeling

# Features Used

- **Categorical (9):** agency, complaint_type, descriptor, borough, location_type, incident_zip, community_board, council_district, police_precinct
- **Numeric (2):** latitude, longitude
- **Time features (4):** created_hour, created_dayofweek, created_month, created_is_weekend



# Preprocessing Pipeline

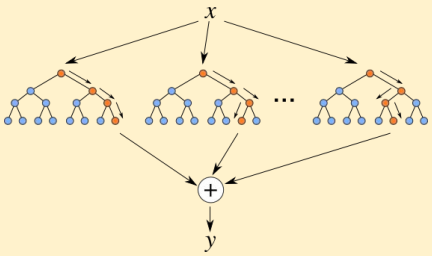
- To avoid leakage, preprocessing is fit **only on training data in each fold**
- Categorical: **most_frequent imputation** + **One-Hot Encoding**
- Numeric/time: **median imputation** + **StandardScaler**
- Libraries Used: Scikit Learn

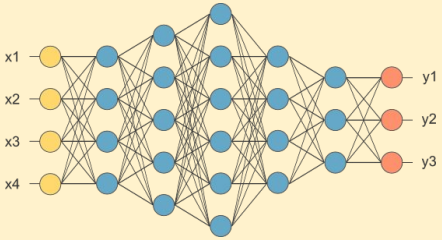
# Evaluation, Cross Validation & Metrics

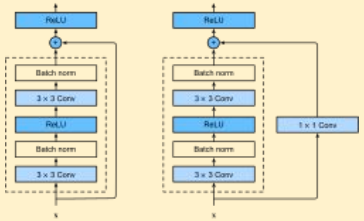
- **5-fold Stratified CV** (stratified by binned target)
- Metrics reported per fold and averaged:
  - **Mean Absolute Error (MAE)**
  - **Root Mean Square Error (RMSE)**
- Best model selected by **lowest mean RMSE** (tie-break by MAE)

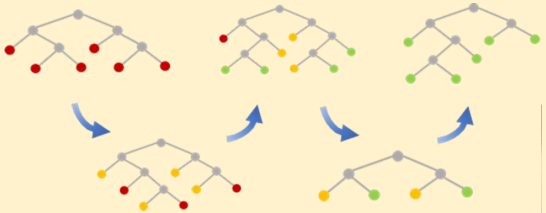
# Models

1. Random Forest Regressor (RFR)
2. Extreme Gradient Boosting (XGBoost)
3. Residual Neural Network (RNN)
4. Multi-layer Perceptron (MLP)



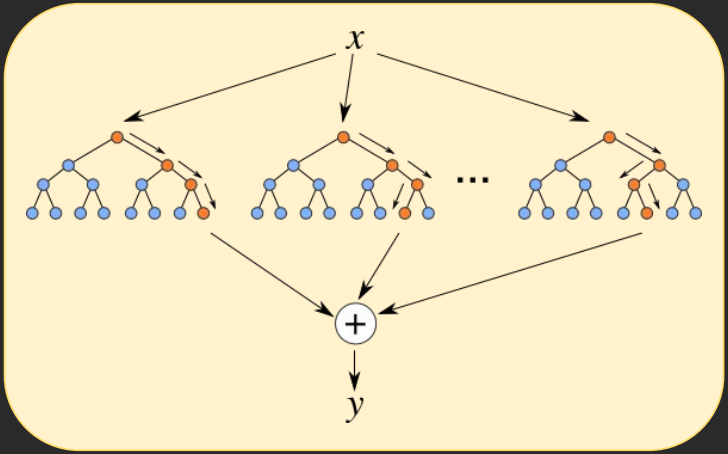



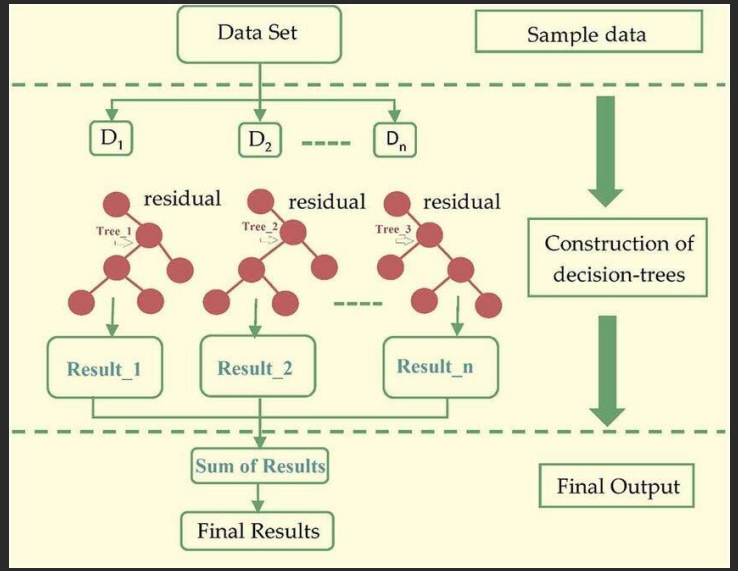
# Model 1: Random Forest Regressor

- **Ensemble learner** of decision trees
- Handles nonlinear interactions and mixed features well
- **Hyperparameters** tuned using grid search:
  - n_estimators ∈ {200, 300}
  - max_depth ∈ {20, 40}
  - min_samples_leaf ∈ {1, 5, 10}
- Library used: Scikit Learn



# Model 2: Extreme Gradient Boosting

- **Ensemble learner** of shallow decision trees
- Hyperparameters tuned using my model:
  - N Estimators ∈ {100, 200}
  - Max Depth ∈ {4, 6}
  - Learning Rate ∈ {0.1, 0.05}
- Library used: Numpy

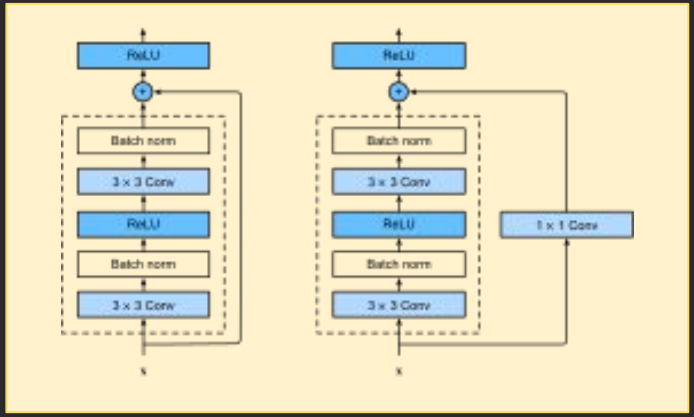


# Model 2a: Extreme Gradient Boosting

- HyperParameters tuned using Pythons XGboost model
  - N Estimators ∈ {100, 500, 800, 1000, 1200}
  - Max Depth ∈ {4, 6, 8, 10, 12}
  - Learning Rate ∈ {0.01, 0.03, 0.05, 0.1, 0.5}
- Library used: xgboost

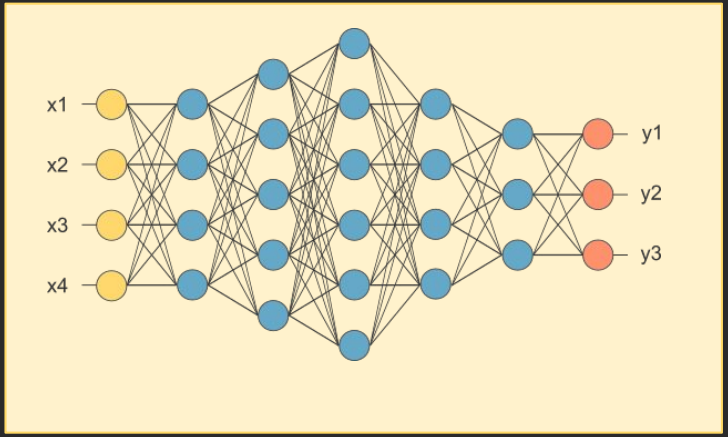
# Model 3: Residual Neural Network

- **Neural network with 2 hidden layers**
- **Hyperparameters** tuned using grid search: Input size = 1252-1255
  - Hidden sizes = [256]
    - No. of blocks ∈ {2, 4}
    - Learning rates ∈ {1e-3, 3e-4}
    - Dropouts ∈ {0.1, 0.3}
    - Batch size = 512
  - Number of epochs = 100
  - Activation function = ReLU
- Libraries used: PyTorch, sklearn



# Model 4: Multi-layer Perceptron

- **Neural network with 2 hidden layers**
- **Hyperparameters** tuned using grid search: Input size = 1252-1255
  - Hidden sizes ∈ {128, 256}
  - Learning rates ∈ {1e-3, 3e-4}
  - Dropout = [0.2]
  - Batch size = 512
  - Number of epochs = 100
  - Activation function = ReLU
- Libraries used: PyTorch, sklearn

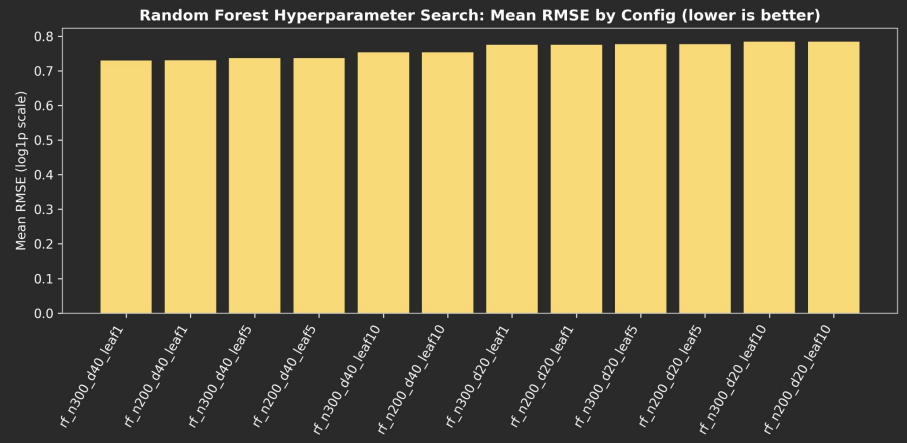


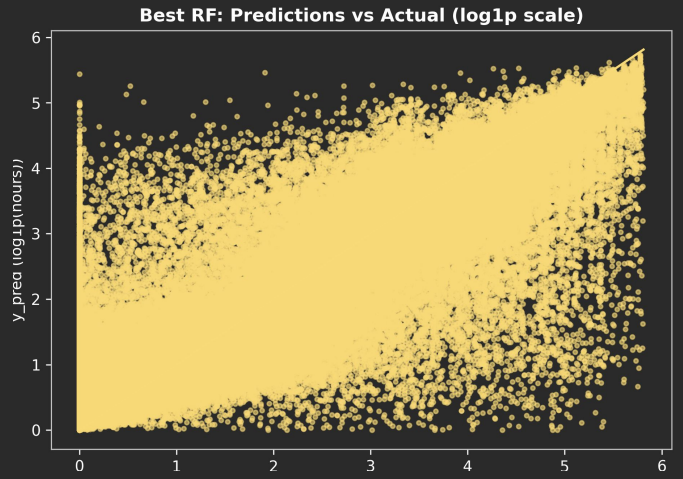
# Results

3

# RF Results

- Best configuration:
  - **rf_n300_d40_leaf1**
    - **n_estimators = 300, max_depth = 40, min_samples_leaf = 1**
  - **MAE**: 0.4648 ± 0.00195
  - **RMSE**: 0.7301 ± 0.00439
- However, they were all close
- Predictions tracked actuals reasonably well on log scale
- Biggest errors occur on extreme long-duration complaints (tail behavior)



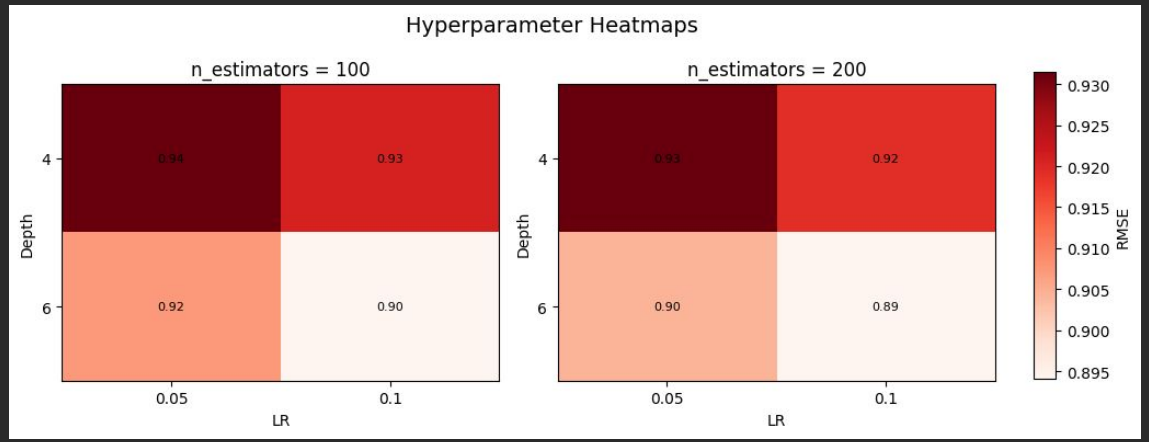
# Python XGBoost Results

| rank | depth | n_estimators | lr | mean_rmse | mean_mae |
|---|---|---|---|---|---|
| 1 | 12 | 800 | 0.01 | 0.860991 | 0.585243 |
| 2 | 12 | 1200 | 0.01 | 0.863091 | 0.577277 |
| 3 | 12 | 500 | 0.01 | 0.863258 | 0.597443 |

- High depth weak learners with a low lr constantly performed the best

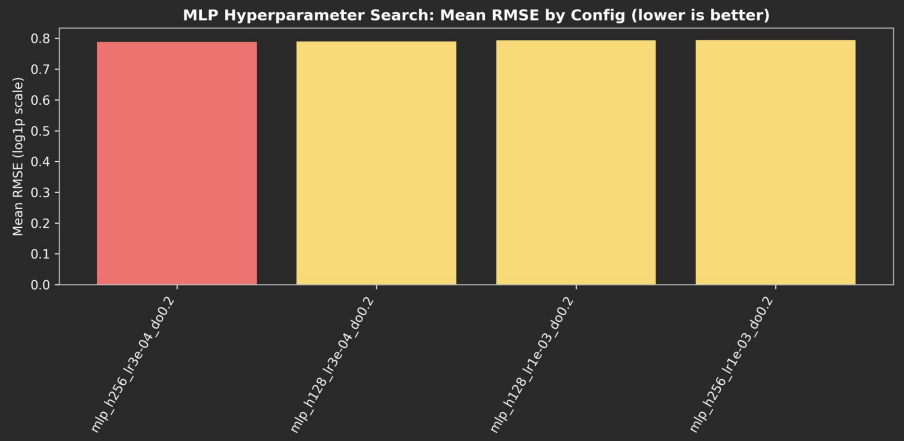
# My XGBoost Results

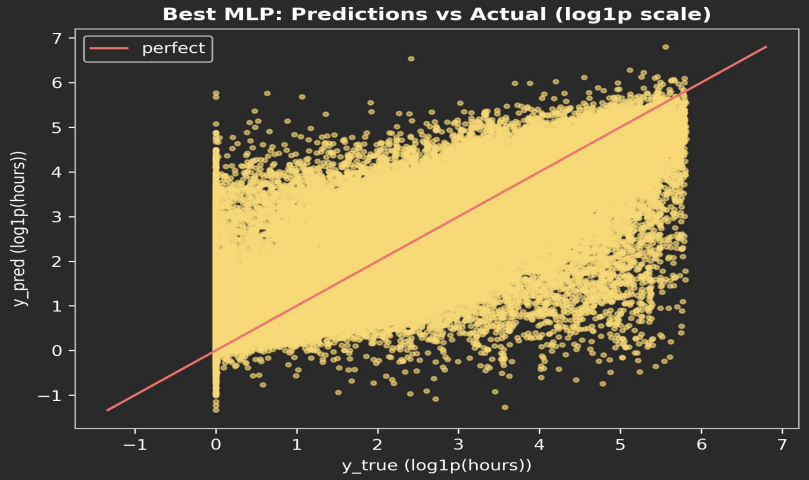
| rank | depth | n_estimators | lr | mean_rmse | mean_mae |
|---|---|---|---|---|---|
| 1 | 6 | 200 | 0.1 | 0.8940 | 0.6384 |
| 2 | 6 | 200 | 0.1 | 0.9042 | 0.6524 |
| 3 | 6 | 100 | 0.1 | 0.9047 | 0.6525 |



# MLP Results

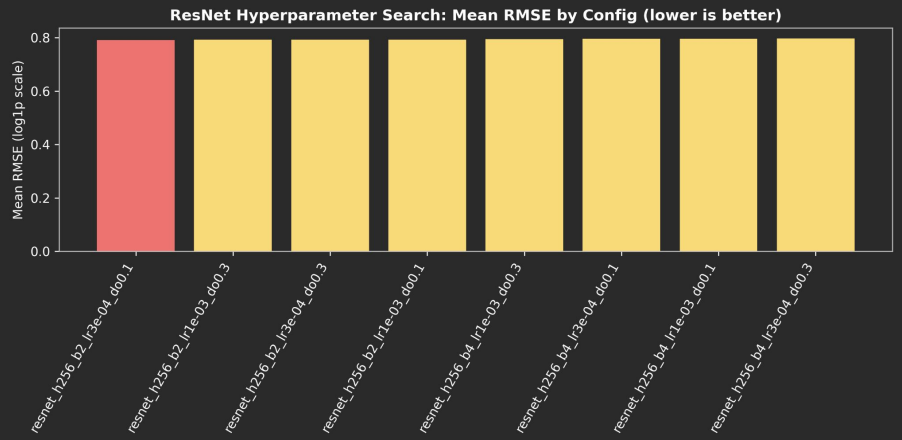
- Best configuration:
  - Hidden layer size = 256
  - Learning rate = 0.0003
  - Dropout = 0.2
  - **MAE**: 0.5463 ± 0.00401
  - **RMSE**: 0.7888 ± 0.00311
- Very small error difference across configurations
- The prediction vs actual log plots show good performance

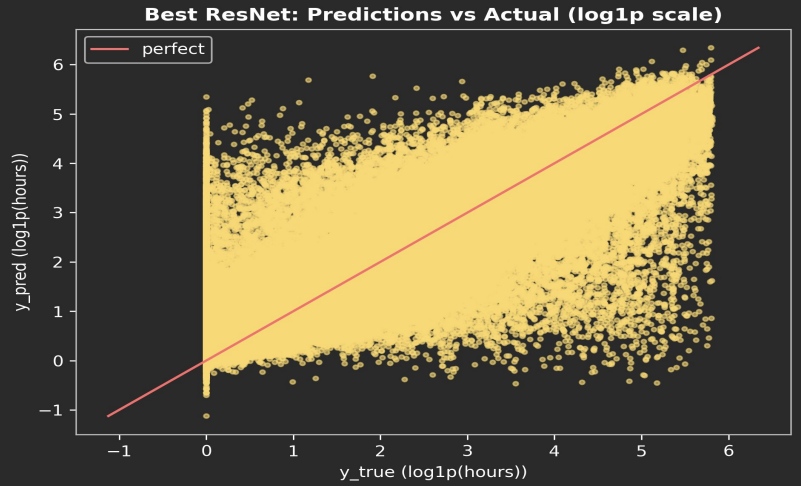


# RNN Results

- Best configuration:
  - Hidden layer size = 256
  - Learning rate = 0.0003
  - Dropout = 0.1
  - No. of blocks = 2
  - **MAE**: 0.5456 ± 0.00400
  - **RMSE**: 0.7911 ± 0.00407
- Small error difference across configurations
- The prediction vs actual log plots show good performance



# Conclusion

4

# Comparing Models

| Model | RMSE | MAE |
|---|---|---|
| Random Forest | 0.7301 | 0.4648 |
| MLP | 0.7888 | 0.5463 |
| RNN | 0.7911 | 0.5456 |
| XGBoost | 0.8609 | 0.5852 |

# Interpretations & Limitations

- **Best performing model:** Random Forest
- **Limitations**:
  - Live dataset (results depend on snapshot used),
  - Small sample of dataset,
  - Minimal hyperparameter tuning,
  - Did not use all features in original dataset
    - descriptions and other free text features would need NLP integration

# Acknowledgements

5

# Collaborators

- Ryan Miner
- Myra Mulongoti
- Obumneme Umeonwuka

# References


- **Dataset:** https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2020-to-Present/erm2-nwe9
- **Random forest image:** https://medium.com/@vk.viswa/surface-level-understanding-of-random-forest-regression-3bac37e06f3e
- **XGBoost Image:** https://www.esri.com/arcgis-blog/products/arcgis-pro/analytics/boosting-the-model-the-enhancement-in-the-forest-based-and-boosted-classification-and-regression-tool
- **MLP Image**: https://www.aretor.it/courses/linguistics/07_mlp/
- **RNN Image:** https://en.wikipedia.org/wiki/Residual_neural_network
- **CSCI-635, Spring 2026 Class Lecture slides**

# Any Questions?

# Thank you for your time.